Command line

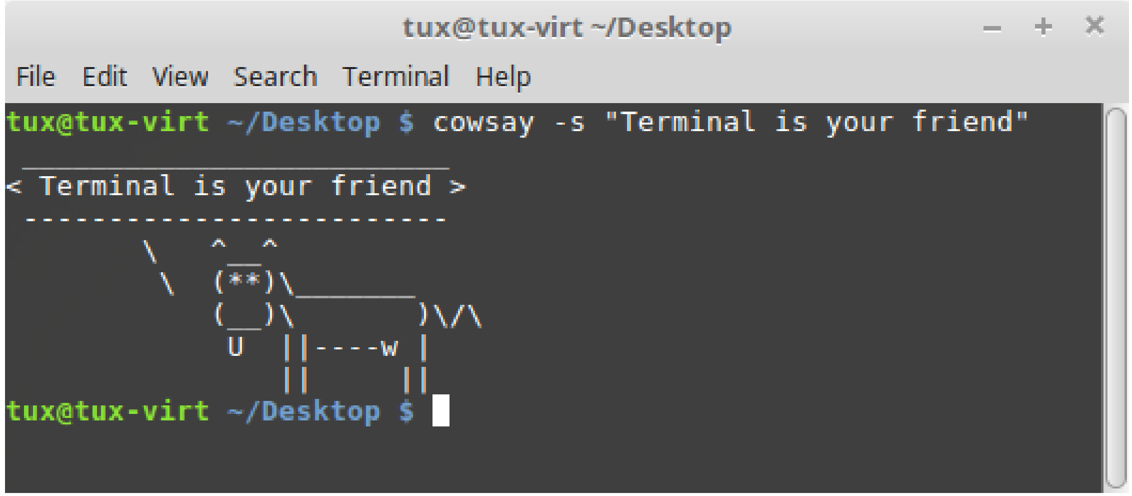
Terminal

GUI is nice to have

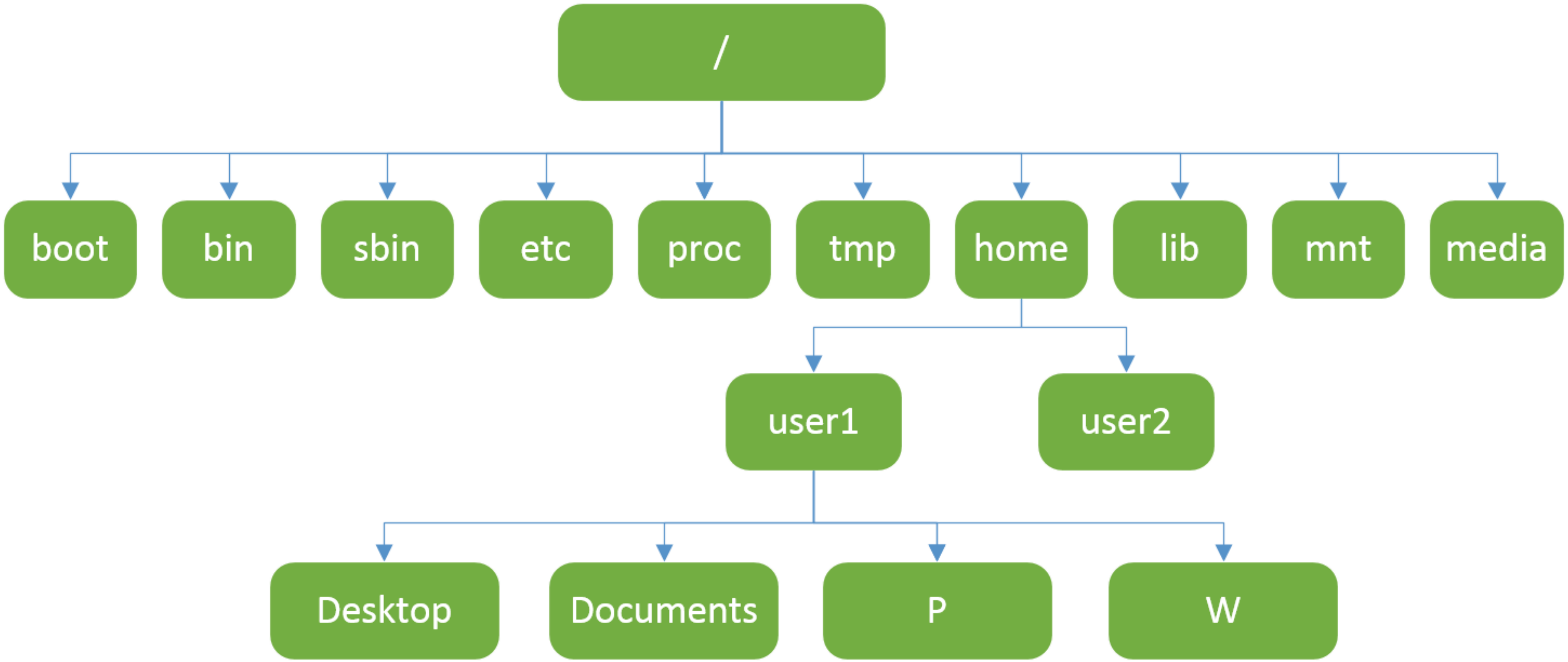
- It’s intuitive (sometimes)
- It’s user-friendly (sometimes)
- Multitasking is easy
- It doesn’t scare people

CLI is a must

- Fast
- Feature-rich
- Big learning curve
- Often the only way



Folder structure



The basics*

You are shown `username@hostname:current_path`

- `e.g. risto.heinsar@lx1:~/P>`

Tab autocompletes (program, file and directory names, some commands within programs)

Up/down arrows navigate history (previous commands)

- Can see a list by typing: `history`

Ctrl + c	Terminates a running application (usually)
Ctrl + d	Send the end of file signal
Ctrl + l	Clears the screen
Ctrl + insert	Copy text
Ctrl + shift + insert	Paste text

* Widely-accepted practices, work on most terminals

SSH connection

SSH (secure shell) is a connection protocol between two computers

- Commonly text-based
- Creating tunnels
- Forwarding ports
- X11
- Underlaying layer for many protocols – SFTP, SCP, rsync, …
- Network drives
- Key based security
- Etc.

`ssh host`	Creates an SSH connection
`ssh user@host`	Specifies a user to connect to
`ssh -X host`	Enables X11 forwarding (graphics)

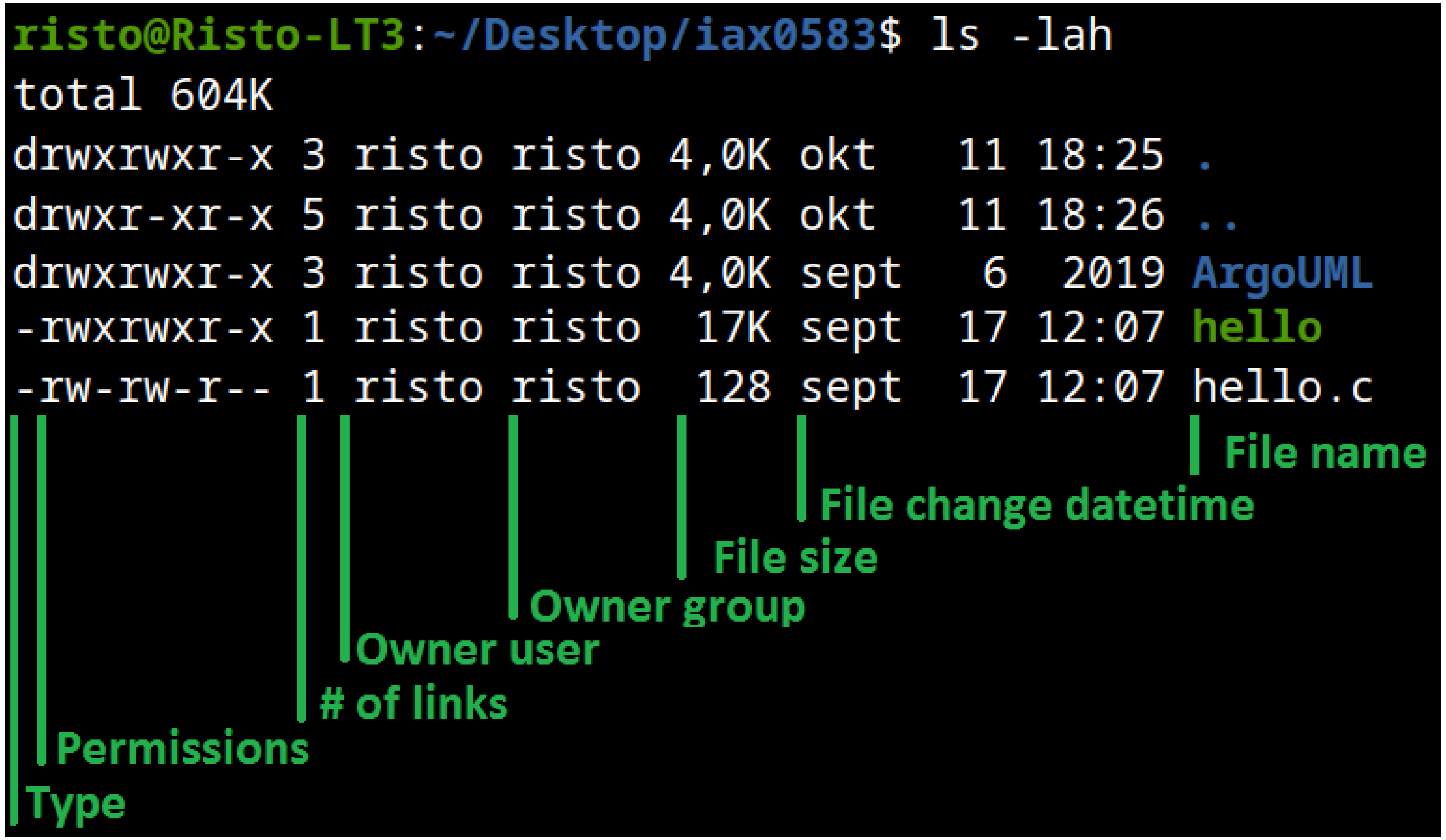
Listing directory contents

`ls`	list directory contents
◦ `ls -l`	list directory with details
◦ `ls -la`	list directory with details, show hidden files and directories
◦ `ls -lat`	list directory with details, show hidden files, order by time modified
◦ `ls /dir`	list the contents of the subdirectory called dir
◦ `ls -l /dir`	list the contents of the subdirectory called dir, shows details
◦ `ls -l *as*.c`	lists all files in current directory that have a .c extension and have the string “as” in the name of the file
◦ …	

Changing current working directory

`cd pr1_files`	move to a subdirectory called pr1_files
`cd ..`	move to the parent directory
`cd ~`	move to the home directory
`cd /`	move to the root directory
`cd -`	move to the previous directory
`cd ~/P/subjects/prog1`	move to the folder specified (full path)
`cd ../..`	move two folders up (relative path)
`cd ../Desktop`	move one folder up and then to Desktop (relative path)

ls output


risto@Risto-LT3:~/Desktop/iax0583$ ls -lah
total 604K
drwxrwxr-x 3 risto risto 4,0K okt   11 18:25 .
drwxr-xr-x 5 risto risto 4,0K okt   11 18:26 ..
drwxrwxr-x 3 risto risto 4,0K sept   6 2019 ArgoUML
-rwxrwxr-x 1 risto risto 17K sept 17 12:07 hello
-rw-rw-r-- 1 risto risto 128 sept 17 12:07 hello.c
File name
File change datetime
File size
Owner group
Owner user
# of links
Permissions
Type

Privileges

Each file has a set of privileges (permissions) for

- User (U)
- Group (G)
- Other (O)

Most common privilege types: read, write, execute (rwx)

`chmod`	to modify privileges
◦ `chmod 777 file`	All permissions for everyone. **Never ever do this unless sure!**
◦ `chmod 740 file`	Owner can do everything, group can read, others have no rights
◦ `chmod u+x file`	Adds execute privilege for file owner, no other permissions altered
◦ `chmod o-rw file`	Removes read and write privilege for other users
◦ `chmod a+r file`	Adds read permission to everyone (UGO)

Other basic commands

`cp`	copy file
`mv`	move / rename file
`rm`	remove file
`mkdir`	create directory
`rmdir`	remove directory
`grep`	find patterns in files
`top`	view processes (`htop` recommended, needs install)
`kill`	kills a process
`sudo`	runs program as root in Debian based systems (including Ubuntu)
`ping`	test latency between devices
`ssh`	Create a secure tunnel to another computer

Man pages

MAN POW

```
tux@tux-virt ~/Desktop
File  Edit  View  Search  Terminal  Help
POW(3)                     Linux Programmer's Manual                    POW(3)

NAME
       pow, powf, powl - power functions

SYNOPSIS
       #include <math.h>

       double pow(double x, double y);
       float powf(float x, float y);
       long double powl(long double x, long double y);

       Link with -lm.

   Feature    Test    Macro    Requirements    for    glibc    (see    fea-
   ture_test_macros(7)):

       powf(), powl():
           _BSD_SOURCE || _SVID_SOURCE || _XOPEN_SOURCE >= 600 ||
           _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L;
           or cc -std=c99

DESCRIPTION
       These functions return the value of x raised to the power of y.

RETURN VALUE
 Manual page pow(3) line 1 (press h for help or q to quit)
```

MAN CP

```
tux@tux-virt ~/Desktop
File  Edit  View  Search  Terminal  Help
CP(1)                            User Commands                           CP(1)

NAME
       cp - copy files and directories

SYNOPSIS
       cp [OPTION]... [-T] SOURCE DEST
       cp [OPTION]... SOURCE... DIRECTORY
       cp [OPTION]... -t DIRECTORY SOURCE...

DESCRIPTION
       Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.

       Mandatory  arguments  to  long  options  are mandatory for short
       options too.

       -a, --archive
              same as -dR --preserve=all

       --attributes-only
              don't copy the file data, just the attributes

       --backup[=CONTROL]
              make a backup of each existing destination file

       -b     like --backup but does not accept an argument
 Manual page cp(1) line 1 (press h for help or q to quit)
```

Viewing files

`cat filename`	Show file contents (basic)
◦ `cat -n filename`	Show file contents, display line number
`more filename`	Goes through a file, one page at a time
`less filename`	More feature-rich version of `more`
`head filename`	Show first 10 lines of a file
`tail filename`	Show last 10 lines of a file

Editing files

Simple editor:

- `nano`

Better and more complex editors:

- `vi`
- `vim`
- `emacs`

To open a file, just type the file name after the program name

- Example: `vim hello.c`

If a file exists, it will be opened. If it doesn't, it will be created.

Compiling

`gcc`	The GNU compiler
◦ `-o filename`	Sets the output file name
◦ `-Wall`	Show “all” warnings
◦ `-Wextra`	Show additional warnings
◦ `-Wconversion`	Show type conversion issues
◦ `-pedantic`	Make it ANSI C compatible
◦ `-lm`	Links the math library
◦ `-std=c90`	Sets C standard to C90 during compilation
◦ Etc.	

Example: compile `mycode.c` source file to a program called `myprogram`

```
gcc -o myprogram -Wall -Wextra -Wconversion mycode.c
```

Running programs

To run your own programs, you need to specify the path to it
(either relative or full)

- `./myprog` program in the same directory as current path
- `../myprog` program in the parent directory
- `/home/usr/Desktop/myprog` program full path

Programs in the PATH environment variable can be executed just by name

- This is why you can run gcc, geany, firefox etc by just typing their name
- `env` list your environment, including PATH

Pipes

You can connect two or more program's input and output streams using a pipe

```
command1 | command2 | command3
```

Sample: Print the list of students who scored in math, ordered by the points. Show only name and points.

```
risto@pc:~$ cat demo | grep Math | sort -k2 -r | cut -d " " -f1,3
Mari 5.0
Toomas 4.6
Siim 4.4
Mati 3.9
```

Contents of demo:

```
Mari Math 5.0
Mari Physics 4.4
Kati Physics 4.8
Kati English 3.9
Siim Math 4.4
Eduard Geography 4.9
Mati Math 3.9
Karin Biology 4.1
Toomas Math 4.6
```